Electronics & Computing Club

Digital Thermometer Project

Build & Code: Arduino Workshop

Written By Nicholas Goryczka on February 20th, 2026

Section 1
Overview

The digital thermometer project will allow anyone to get to implement one of the most important building blocks in microcontroller programming: reading sensor values, processing the data, and outputting it in a way that a user or operator can understand. So what better sensor than a temperature sensor, a sensor so ubiquitous there's multiple in everyone's computers, phones, and more.
By the end of this little project, you'll be able to measure temperature precisely through the console and output a rounded value to the bar LED.

I would recommend this project with anyone trying to get started with C programming for the first time, since this project is mostly straightforward.

Section 2
Required Components

You will need the following components:

- 10x 330Ω resistors
- 1x Bar led array (10 LEDs)
- 1x LM35 temperature sensor

Section 3

Circuit Assembly

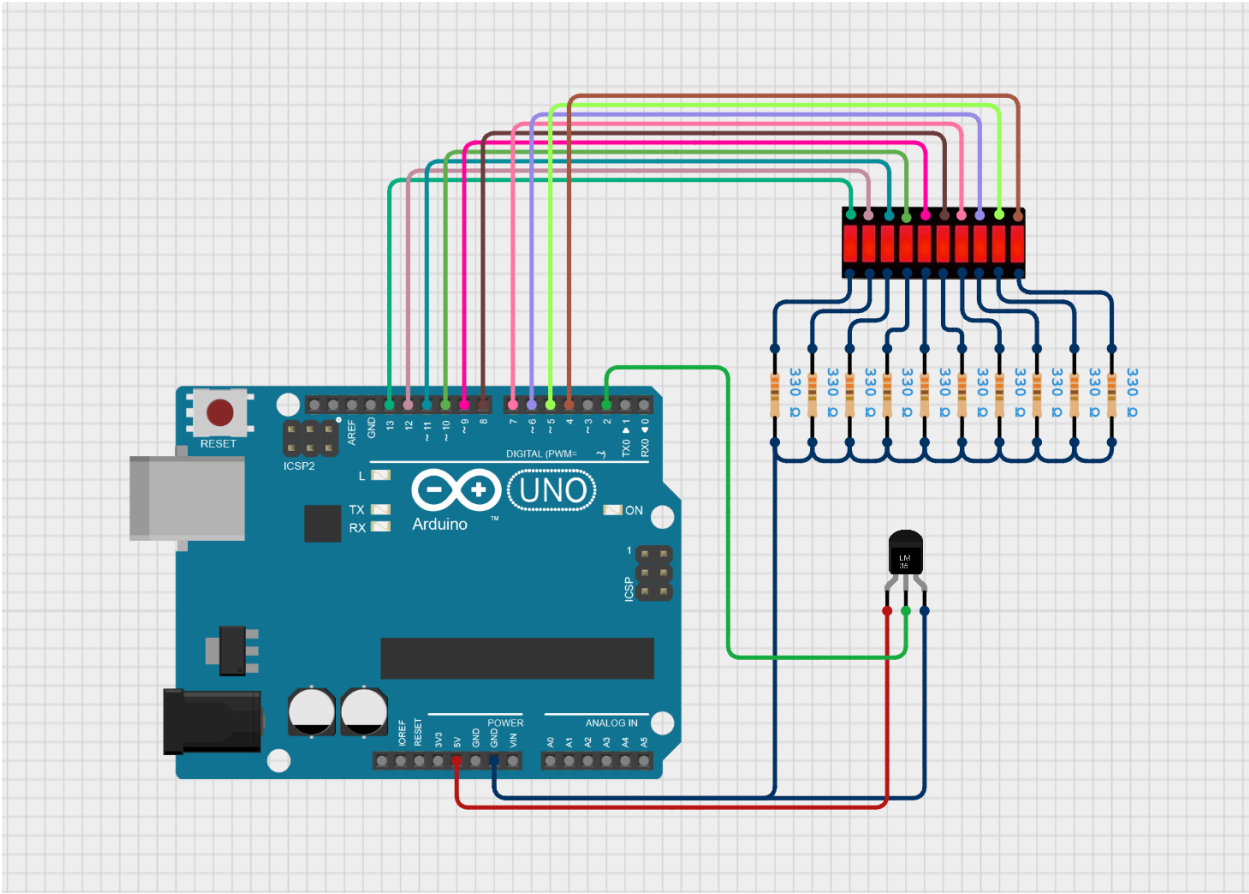


Figure 1 Circuit schematic. Credit: Charles Ross
https://app.cirkitdesigner.com/project/be5bfbb1-5a3c-4950-b1ad-5075e7993201

Above, we can see the circuit schematic, which tells us in detail which component goes to which pin on the Arduino. There is also a link included if you need a higher resolution and interactive view of it.

If you haven't had previous experience working with electronics or prototyping with electronics on a breadboard, it is strongly recommended you go check out the *Breadboard Guide* before continuing.

Bar LED pack

This one is a funny one for how simple it is in construction. So, I assume you've at least heard of the term LED, which stands for Light Emitting Diode. These allow for nice and efficient light output than all other commercially viable technologies.

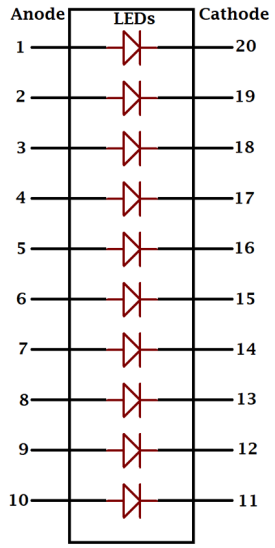


Figure 2 Abstract view of the internals of a bar LED pack Credit: Components101.com

I want you to take notice that these are in fact diodes, meaning that they work in a very interesting way. You can think of a diode like a one-way turnstile: it will only let you go through one way, and you can't try and go through it in the opposite manner, you need to put a bit of effort as you pass through it. Therefore, a diode allows energy to flow in one direction and in turn takes some voltage; that is the trade-off. In the case of LEDs, that spent energy is turned into light.

You can see in Figure 2, that the diode symbols are like arrows with a bar, that tells us the direction of energy: Anode is positive and Cathode is negative.

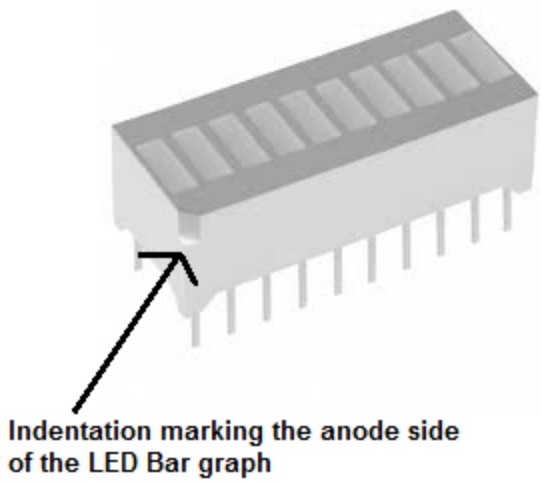


Figure 3 What to look for when trying to find the Bar LED anode

Unfortunately, it's likely that the bar LEDs have a much more subtle indicator to tell us which side is the anode(+), but it should still be almost the same as the picture. The only difference being, the notch is present on the corner from top to bottom instead of halfway down.

Resistors

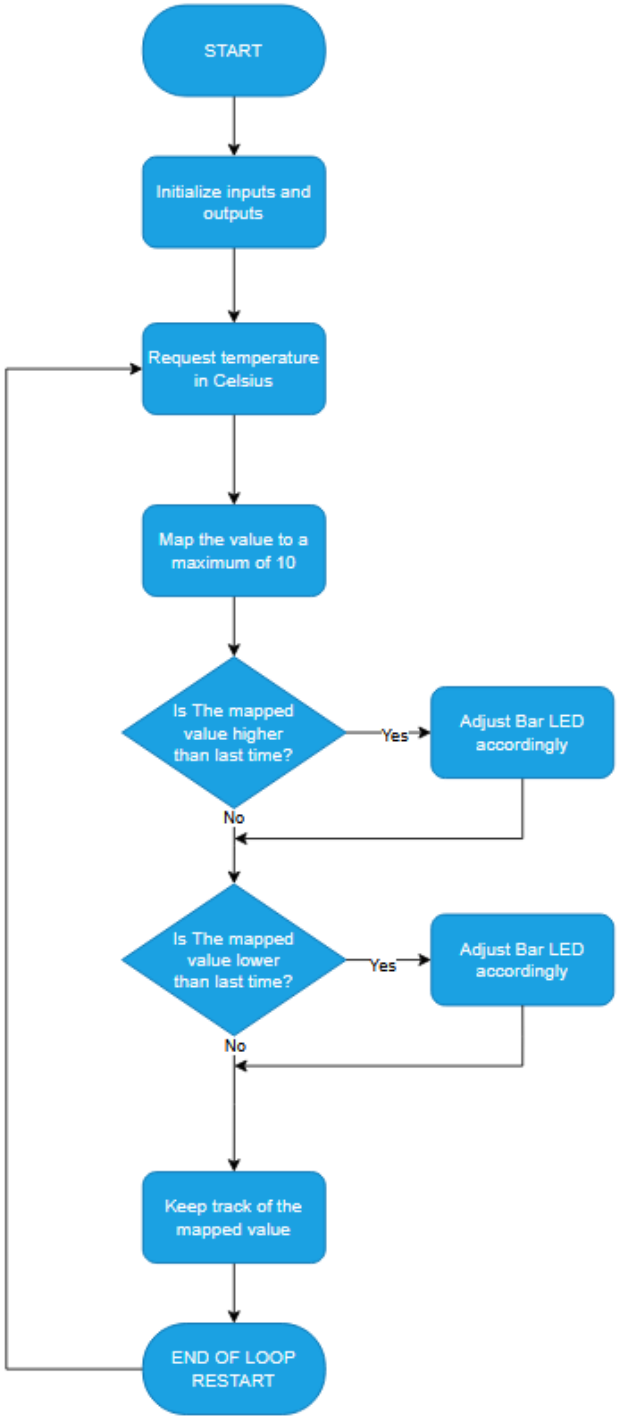
The most ubiquitous electronic component, simple and straightforward: the resistor. These are here to limit current. I'd recommend to look at page 4 of the *Color Mixer Project*, because Alexander does a wonderful job explaining it. In short, you can think of the three elements: Voltage (V), Current (I), Resistance (R), as ways one can influence the flow of electrical energy. One simply needs to remember Ohm's law:

$$V = IR \qquad I = \frac{V}{R} \qquad R = \frac{V}{I}$$

I bring this up, because diodes do not directly limit current, they only cause a very minimal step down of voltage. This indirectly affects current, but it does not limit it, the diode does not dissipate excess current that passes through it.

This is why we need to put resistors on the cathode side of the Bar LED, so that remaining current is dissipated into heat, making sure that all that remaining energy isn't sent directly into the Arduino, because then we fry it.

Section 4
Coding



The flowchart on the previous page will guide you through coding.

Starting from the top, you'll need to include two libraries. These will allow you to interface with the temperature sensor and the Serial console.

```
#include <OneWire.h>
#include <DallasTemperature.h>
```

Now that the libraries are included, we can move on to defining which pins we're going to use. C language allows us to declare data types, as well as put modifiers onto said data types.

```
const int tempPin = 2;
const int barLEDPins[10] = {13, 12, 11, 10, 9, 8, 7, 6, 5, 4};
```

Above, we have two interesting variable declarations. Firstly, both of these are constant with the use of the *const* modifier. Secondly, the barLEDPins value has square brackets next to it and is declared in a very odd manner; this is what we call an array. Arrays are very interesting data types. They are blocks of memory that are accessed by offsetting with size and using a single frame of reference.

Arrays always start at 0, and end at 1 less than their size. The 0th element is the frame of reference.
A good way to think of arrays is with the following 2 scenarios: your good friend asks to see one of your neighbors. You tell him that they are 3 houses down the street.
You and your friends are staying in a hotel, occupying 2 rooms. The friend you're staying with keeps forgetting where the other room is, you keep telling him that it's 5 rooms down the hall.

In these examples, when we were giving directions, we specified sizes: houses and rooms, and we provided an offset: 3 houses down and 5 rooms down.

Next up, still remaining outside of either setup() or loop(), we need to add another variable, but this time, dynamic.

```
int previousBarLevel = 0;
```

This value will store the last value of the LED bar so that we can adjust our LED display.

After that, we need to declare possibly the two most important things here, the classes for wire communication

```
OneWire oneWire(tempPin);
DallasTemperature sensors(&oneWire);
```

You'll notice how similar the names are to the libraries, that's because these are from there. These two will let us interface with the thermometer; we'll see a little more about these later.

Now entering the setup() function, we need to set pin modes: the tempPin and barLEDPins. This is where the *pinMode()* comes in

```
pinMode([pin number], {OUTPUT/INPUT});
```

Setting the tempPin should be straightforward enough, but barLEDPins might be a bit tricky.

```
for (int x = 0; x <= 9; x++)
{
    pinMode(barLEDPins[x], OUTPUT);
}
```

Here is a simple for() loop, that handles setting all the pins within barLEDPins. It does this by using the integer *x* as an iterator. It starts at 0, sets the 0th element of barLEDPins as an output (Pin 13), and then increments.
Now at *x*=1,it sets the 1st element of barLEDPins as an output (Pin 12). It then continues doing this until it reaches the 9th element. Once it does the same song and dance, then x reaches the value of 10. Now that *x* is outside of the condition in the *for*() loop, the program leaves the loop.

```
Serial.begin(9600);
sensors.begin();
```

Above, we are setting up the seriall interface and the sensors. Remember that *Serial* is part of the base code for Arduino, while *sensors* is part of the DallasTemperature library we included.

Finishing off with the code body for the *loop*() function, we need to do some things that may not be so obvious. We first have to tell the Arduino to check and see that our temperature sensor is connected and intact.

```
sensors.requestTemperatures();
```

After that, we can then actually read the temperature from the sensor with the following:

```
sensors.getTempCByIndex(0);
```

We supply 0 as a function parameter, because that is the index of the sensor that we got with the function right before. It follows like an array.
Remember that this function returns a value, therefore you need to make sure that you write this to an actual variable like an int.

Now that we have a temperature, we need to map it to the 10 LEDs we plan to use as outputs. For that, we have the *map*() function:

```
int adjusted = map(input, input_min, input_max, out_min, out_max);
```

So we can see that input is the value that we are trying to map from. The values *input_min* and *input_max* are the input's range before being adjusted. The values *out_min* and *out_max* are the output's range that we want after adjustment.

An example would be input being equal to 200. Then input_min is 0 and input_max is 255. Then, out_min would be 0 and out_max would be 127; effectively half of the input. Therefore, adjusted's value would be 100.

Finally, there is handling the LED bar output. To do this requires a little bit of what might feel daunting at first.

```
for (int x = before; x <= barLevel; x++)
{
      digitalWrite(barLEDPins[x], HIGH);
}
```

So here, we start at the stored barLevel, which would be at 0 during the first run, write to the corresponding bar LED, and then continue after incrementing x by one. We need to do this for both increases and decreases in temperature.

Finally, if your code is acting weird, make sure there's a *delay*() in the *loop*() function. Any other necessary things should be in the *Programming Library*. Good luck and have fun!